

循环神经网络截图详解

为什么上半部分有记忆，下半部分没有

2026 年 4 月 9 日

目录

1 极简背景：为什么要引入隐藏状态	2
2 核心逻辑：循环部分到底在干什么	2
3 把下半部分公式拆开看	3
3.1 第一层：每个符号分别是什么意思	3
3.2 第二层：从输入矩阵到隐藏特征，到底做了哪三步	4
3.3 第三层：从隐藏特征到输出，又在做什么	7
3.4 第四层：为什么它没有“记忆”	7
4 具体栗子：把整个前馈流程代成数字看一遍	8
4.1 先算隐藏层	8
4.2 再算输出层	9
4.3 为什么这就解释了 RNN 的必要性	10
5 困惑度：语言模型到底有多“拿不准”	10
5.1 为什么会需要困惑度	10
5.2 先别看公式，先看直觉	10
5.3 公式到底在做什么	11
5.4 一个真正算得出来的例子	12
5.5 再看一个差模型	12
5.6 为什么有人说它像“平均分支数”	13
5.7 极端情况最能说明问题	13
5.8 最后一句话记住它	13

6 通过时间反向传播：误差是怎样沿时间倒着传回去的	13
6.1 为什么普通反向传播在这里不够	13
6.2 先用一句人话概括 BPTT	14
6.3 黑盒视角：BPTT 到底做哪几步	14
6.4 一个三步序列的直观例子	14
6.5 为什么图里会出现递推求导	15
6.6 为什么会出现梯度消失和梯度爆炸	18
6.7 为什么实践里常说 truncation 和 detach	20
6.8 这张图到底在回答什么真实问题	20
6.9 先把截图里的出场对象翻译成成人话	20
6.10 最小贯穿例子：为什么会出现“追责多远”的问题	21
6.11 上半部分：三种策略到底在干什么	22
6.12 下半部分：为什么突然写出一组线性公式	23
6.13 再看 8.7.2：这张依赖图到底在算什么	24
6.14 把整张图压成一句最短的话	27
6.15 对比：这张图没有在讲什么	27
6.16 最后一句话记住 BPTT	28
7 小结	28

1 极简背景：为什么要引入隐藏状态

这张图真正想解决的是一个很现实的问题：处理序列时，当前词 x_t 的意义，往往依赖前面已经出现过的内容。若把全部历史 $x_1, \dots, x_{t-1}$ 都原样存下来，存储会越来越重，模型规则也会越来越复杂。

因此图里写出

$$P(x_t \mid x_{t-1}, \dots, x_1) \approx P(x_t \mid h_{t-1}).$$

它不是说“历史不重要了”，而是说“历史先被压缩成一个短小的摘要 h_{t-1} ”。这个摘要就叫隐藏状态。它的角色很像工作记忆：不把过去逐字背下来，而是留下对下一步最有用的信息。

2 核心逻辑：循环部分到底在干什么

递推式

$$h_t = f(x_t, h_{t-1})$$

只表达一件事：每到一个新时刻，模型都把当前输入和旧记忆合起来，生成新记忆。

所以循环神经网络真正特别的地方，不是“多了一层 hidden”，而是“多了一个沿时间传递的状态”。这让模型不是每一步都从头开始，而是带着前文留下的摘要继续往前算。

RNN: 带有序列记忆

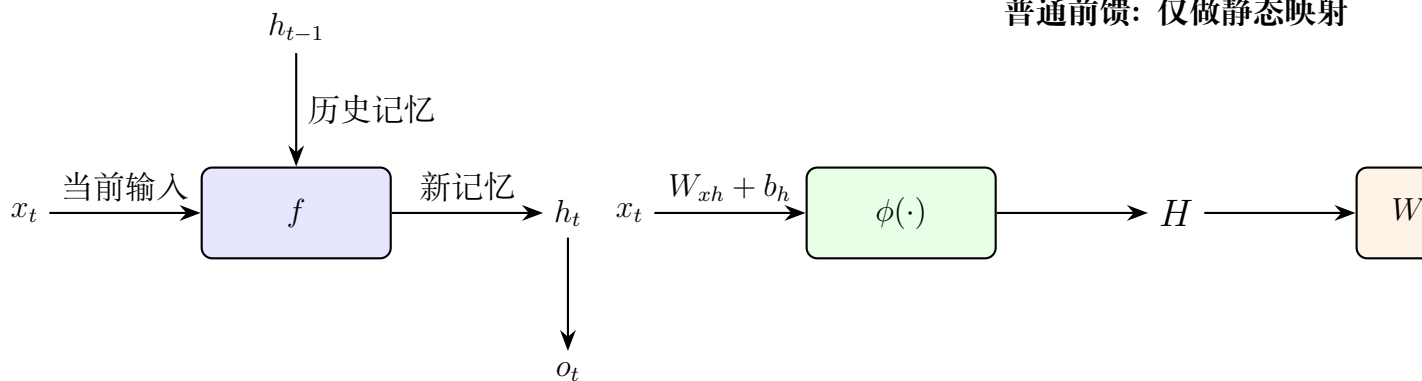


图 1: 循环神经网络 (左) 与普通前馈网络 (右) 的结构对比

3 把下半部分公式拆开看

你问的正是截图下半部分:

$$H = \phi(XW_{xh} + b_h), \quad O = HW_{hq} + b_q.$$

这部分本质上不是循环, 而是普通前馈网络。下面按最朴素的顺序把它拆开。

3.1 第一层: 每个符号分别是什么意思

先约定维度:

- $X \in \mathbb{R}^{n \times d}$: 输入矩阵;
- n : 一批样本里有多少个样本;
- d : 每个样本有多少个输入特征;
- $W_{xh} \in \mathbb{R}^{d \times h}$: 从输入层到隐藏层的权重;
- $b_h \in \mathbb{R}^{1 \times h}$: 隐藏层偏置;
- $H \in \mathbb{R}^{n \times h}$: 隐藏层输出, 也就是中间特征;
- $W_{hq} \in \mathbb{R}^{h \times q}$: 从隐藏层到输出层的权重;
- $b_q \in \mathbb{R}^{1 \times q}$: 输出层偏置;
- $O \in \mathbb{R}^{n \times q}$: 最终输出分数。

所以这两行式子, 其实就是一句非常普通的话:

先把输入 X 变成中间特征 H , 再把中间特征 H 变成输出 O 。

3.2 第二层：从输入矩阵到隐藏特征，到底做了哪三步

看第一条式子：

$$H = \phi(XW_{xh} + b_h).$$

它可以机械地分成三步。

为了不让这三步悬在空中，我们先立刻代一个**带语义的最小例子**。假设我们现在看的是一个固定输入片段 `not happy`，并且先人为抽出两个输入特征：

- 第一个特征：正向词强度；
- 第二个特征：否定词强度。

为了避免“这个数字为什么是 2 而不是 3”这种干扰，我们故意采用最朴素的**计数式编码**：

- x_1 = 这个片段里正向词出现了几次；
- x_2 = 这个片段里否定词出现了几次。

于是对 `not happy`，我们就直接写成

$$x = [1, 1], \quad W_{xh} = \begin{bmatrix} 1 & 1 \\ -2 & 1 \end{bmatrix}, \quad b_h = [0, -1].$$

这里的数字都有一个很具体的解释：

- $x = [1, 1]$ 表示：这个片段里有 1 个正向词 `happy`，也有 1 个否定词 `not`；
- W_{xh} 的第一列 $[1, -2]^T$ 对应第一个隐藏单元，它想测的是“**支持正面的证据减去反对正面的证据**”，前文为了简写，把它叫作“净正向程度”；
- W_{xh} 的第二列 $[1, 1]^T$ 对应第二个隐藏单元，它想测的是“**情绪是否明显被激活**”。

因此， x 不是从天上掉下来的神秘向量，而是我们先用人能理解的规则，把短语翻译成一个很简单的特征表。然后矩阵乘法再在这个特征表上做进一步加工。

这里的“净正向”并不是什么额外术语，它只是一个口语化缩写，意思就是：

一句话里，支持“这是正面的”那部分证据，减去反对“这是正面的”那部分证据，最后剩下多少。

可以把它想成一个很粗糙的记分牌：

- `happy`：支持正面的证据多，反对正面的证据少，所以“净正向”偏高；
- `not happy`：虽然有正向词 `happy`，但又有 `not` 在反对它，所以“净正向”会被拉低；
- `terrible`：几乎没有支持正面的证据，反而全是反向证据，所以“净正向”很低。

因此，“净正向”不要理解成一个新概念，而要理解成一句极朴素的话：

这句话最后到底还剩下多少正面味道？

这里还要补一句最关键的话： W 和 b 的数字大小，本质上是在表达“这个特征会把某个隐藏单元往哪个方向推、推得多强”。

- **权重的正负号**。正号表示“这个特征出现时，会把该隐藏单元的分数往上推”；负号表示“这个特征出现时，会把该隐藏单元的分数往下压”。
- **权重的绝对值**。绝对值越大，说明这个特征的影响越强。比如第一列中的 -2 比 $+1$ 的绝对值更大，表示在这个玩具模型里，“否定词”对净正向程度的破坏，比“正向词”对净正向程度的提升更强。
- **偏置 b** 。偏置表示这个隐藏单元在没有足够证据时的默认起点。负偏置相当于提高激活门槛，正偏置相当于降低激活门槛。

所以，在这个例子里：

- 第一列 $[1, -2]^T$ 的意思不是“随便写两个数”，而是“一个正向词记 $+1$ ，一个否定词记 -2 ，说明否定作用更强”；
- 第二列 $[1, 1]^T$ 的意思是“无论是正向词还是否定词，只要情绪相关词出现，都会把这个单元往上推”；
- 偏置 $[0, -1]$ 的意思是“第二个单元默认先扣 1 分，所以只有情绪证据足够明显，它才真正亮起来”。

第一步：线性变换。 先算 XW_{xh} 。这一步的作用，是把每个样本原本的 d 个特征，重新组合成 h 个新的特征。你可以把它理解成“换一个观察角度看输入”。

在这个具体例子里，

$$xW_{xh} = [1, 1] \begin{bmatrix} 1 & 1 \\ -2 & 1 \end{bmatrix} = [-1, 2].$$

这里的两个数字不是凭空来的，而是按行乘列算出来的：

$$-1 = 1 \cdot 1 + 1 \cdot (-2), \quad 2 = 1 \cdot 1 + 1 \cdot 1.$$

现在这两个结果就有意义了：

- 第一个隐藏单元得到 -1 ，意思是：虽然有一个正向词 **happy**，但它又被一个否定词 **not** 压了回去，所以“净正向程度”偏负；
- 第二个隐藏单元得到 2 ，意思是：这句话里确实同时出现了情绪词和否定词，因此情绪活动是明显的。

所以，原始输入的两个特征，并不是被原样搬过去，而是被重新组合成了两个更贴近任务语义的隐藏特征。

第二步：加偏置。 再加上 b_h 。偏置的作用，是让每个隐藏单元都有一个自己的基准点，而不是必须围绕零点工作。

在这个例子里，就是

$$[-1, 2] + [0, -1] = [-1, 1].$$

你可以把偏置理解成：在线性变换之后，再整体平移一下每个隐藏单元的起点。

这里第二个单元减去 1，可以理解为一种门槛设置：只有情绪信号足够明显，它才会真正被激活。

如果把第二个隐藏单元单独拿出来看，它的线性部分其实就是

$$x_1 \cdot 1 + x_2 \cdot 1,$$

也就是把“正向词个数”和“否定词个数”加起来。再算上偏置 $b = -1$ ，并接上 ReLU 之后，就变成

$$\text{ReLU}(x_1 + x_2 - 1).$$

这时，所谓“默认先扣 1 分”，意思就非常具体了：

- 若证据总量是 0，输出是 $\text{ReLU}(-1) = 0$ ；
- 若证据总量是 1，输出是 $\text{ReLU}(0) = 0$ ；
- 若证据总量是 2，输出是 $\text{ReLU}(1) = 1$ 。

所以，这个偏置不是在制造神秘感，而是在设一个**激活门槛**：

只有当情绪相关证据的总量超过 1 以后，这个隐藏单元才会真正亮起来。

用三个最短的片段对比，会更直观：

片段	$x = [x_1, x_2]$	线性分数 $x_1 + x_2$	加偏置后 $x_1 + x_2 - 1$	ReLU 后 输出	解释
happy	[1, 0]	1	0	0	只有一点情绪证据，还不算明显
not	[0, 1]	1	0	0	也只有一点证据，仍未真正激活
not happy	[1, 1]	2	1	1	证据累计够多，单元真正亮起

这张表想说明的不是“这些数字永远正确”，而是偏置 $b = -1$ 在做什么：

它把“有一点证据就亮”改成了“证据累计到一定程度才亮”。

第三步：过激活函数。 最后对每个元素施加 ϕ ，比如 ReLU 或 tanh。这一步的意义，是让网络不只是“直线变换”，而能够表达更复杂的非线性关系。

若这里取 ReLU，那么负数会被截成零，正数保持不变，所以

$$H = \text{ReLU}([-1, 1]) = [0, 1].$$

于是你现在终于能把这一整条式子看成一个非常具体的过程：

$$[1, 1] \longrightarrow [-1, 2] \longrightarrow [-1, 1] \longrightarrow [0, 1].$$

这四个向量依次对应：

- 原始输入；
- 线性变换后；
- 加偏置后；
- 过激活函数后，也就是最终的隐藏特征 H 。

做完这三步以后，得到的 H 就是隐藏层表示。这里的“隐藏”，只表示它是网络内部的中间特征，不表示它具备跨时间的记忆能力。

3.3 第三层：从隐藏特征到输出，又在做什么

第二条式子

$$O = HW_{hq} + b_q$$

表示把隐藏特征进一步映射到输出空间。

若任务是分类，那么 O 往往是每一类的分数，后面再接 softmax 变成概率。若任务是回归，那么 O 也可能直接就是预测值。

因此，下半部分整体流程只有这一条主线：

$$\text{输入} \longrightarrow \text{隐藏特征} \longrightarrow \text{输出}.$$

这就是最标准的前馈结构。

3.4 第四层：为什么它没有“记忆”

真正关键的是，这组公式没有出现 h_{t-1} 这样的量。也就是说：

- 当前输出不显式依赖上一个时刻的状态；
- 网络没有一个专门的“滚动笔记本”在时间上往下传；
- 每个输入样本进来时，都是独立做一次映射。

所以，虽然这里也有一个字母 H ，但它不是循环神经网络里“记忆过去”的隐藏状态，而只是普通神经网络中的中间层输出。

这正是上半部分和下半部分的根本差别：

- 上半部分： $h_t = f(x_t, h_{t-1})$ ，重点是**记忆会往后传**；
- 下半部分： $H = \phi(XW_{xh} + b_h)$ ， $O = HW_{hq} + b_q$ ，重点是**当前输入做一次静态映射**。

4 具体栗子：把整个前馈流程代成数字看一遍

上面我们已经把 $X \rightarrow H$ 的三步单独算过一次。现在把后半句

$$O = HW_{hq} + b_q$$

也接上去，看看完整前馈链条是怎样结束的。仍然看同一个片段 `not happy`，仍然只看一个样本。这里依旧使用刚才那套人工定义的计数式特征：

$$x = [1, 1].$$

再设

$$W_{xh} = \begin{bmatrix} 1 & 1 \\ -2 & 1 \end{bmatrix}, \quad b_h = [0, -1].$$

激活函数取 ReLU。

4.1 先算隐藏层

第一步，线性变换：

$$xW_{xh} = [1, 1] \begin{bmatrix} 1 & 1 \\ -2 & 1 \end{bmatrix} = [-1, 2].$$

第二步，加偏置：

$$[-1, 2] + [0, -1] = [-1, 1].$$

第三步，过 ReLU：

$$H = [0, 1].$$

这说明：原始输入 $[1, 1]$ 被重新加工成了一个新的隐藏表示 $[0, 1]$ 。这一步和上一节的小例子完全一致。它的语义是：

- “净正向程度”这个隐藏单元被压成了 0，说明它没有留下明显正向证据；
- “情绪明显存在”这个隐藏单元保留为 1，说明这句话确实带有情绪色彩。

4.2 再算输出层

现在再取

$$W_{hq} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad b_q = [0, 0].$$

我们约定：

- 第一个输出分量代表“正面分数”；
- 第二个输出分量代表“负面分数”。

这里最好也像前面解释 W_{xh} 一样，把 W_{hq} 的数字赋予明确语义。因为现在 $H = [0, 1]$ 有两个分量：

- 第一个隐藏特征：净正向程度；
- 第二个隐藏特征：情绪明显存在。

于是，输出层权重矩阵 W_{hq} 的每一列，都可以看成“某个类别自己的打分规则”：

- 第一列 $[1, -1]^T$ 是“正面类”的打分规则；
- 第二列 $[-1, 1]^T$ 是“负面类”的打分规则。

这两列的含义分别是：

- 对正面类而言，净正向程度越高越好，所以给第一个隐藏特征乘 +1。但“情绪很明显”这件事本身，并不等于“这是正面情绪”。一句话也可能是强烈的负面，比如 **terrible**、**awful**。所以如果一个片段只是说明“情绪很强”，却没有留下正向证据，那么它并不能单独支持正面类，因此第二个隐藏特征在正面列里被乘上 -1。
- 对负面类而言，若净正向程度不高，且情绪又很明显，那么更像负面，因此它对第二个隐藏特征给 +1，对第一个隐藏特征给 -1。

所以， W_{hq} 的数字大小也不是随便写的：

- 正号表示“这个隐藏特征会提高该类别分数”；
- 负号表示“这个隐藏特征会压低该类别分数”；
- 绝对值越大，说明这个隐藏特征对该类别越重要。

这个例子里我们把绝对值都设成 1，只是为了让计算尽量透明。真实模型里，这些数往往不相等，它们会反映“某个隐藏特征对某个类别到底有多关键”。

于是

$$O = HW_{hq} + b_q = [0, 1] \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} = [-1, 1].$$

这个乘法如果按“看列打分”的方式来读，会更直观：

- 正面分数 = -1。因为当前没有留下正向证据，但“情绪明显存在”这个特征却在正面列里带着负号，所以正面分数被压低了。
- 负面分数 = 1。因为当前没有明显正向证据，同时“情绪明显存在”这个特征在负面列里带着正号，所以负面分数被抬高了。

因此，结果 $[-1, 1]$ 不是“随便出现一个负数和一个正数”，而是在表达：

按这套打分规则，这个片段对“负面类”的支持，高于对“正面类”的支持。

如果后面再接 softmax，它就会把这两个分数变成概率；但在 softmax 之前，我们已经能从大小关系上读出：这个固定片段更像负面，而不是正面。

整套过程到这里就结束了。你会发现，它从头到尾都只处理当前这个输入片段，没有任何一个位置专门存放“上一时刻发生了什么”。

4.3 为什么这就解释了 RNN 的必要性

假设当前词是 happy。若只用下半部分这类前馈公式，模型主要根据“当前输入向量”来判断，很难自然表达“前面出现过 not，所以这次的 happy 不该按正面理解”。

而 RNN 的上半部分会把前面信息先压进 h_{t-1} ，再在当前步和 x_t 一起使用。于是“前面说过什么”就可以真正影响“现在怎么解释”。

5 困惑度：语言模型到底有多“拿不准”

5.1 为什么会需要困惑度

做语言模型时，我们真正关心的问题其实很朴素：

模型看到前文以后，对下一个词到底猜得准不准？

如果模型面对前文

```It is raining''`

时，能把高概率给到 outside、today 这类合理续写，而把很低概率给到 banana tree 这种明显奇怪的续写，那么我们会说这个模型更好。

麻烦在于，语言模型输出的不是一句“我觉得对”或“我觉得错”，而是一整张概率表。于是我们需要一个数字，把“模型到底有多确定”压缩出来。困惑度就是这个数字。

### 5.2 先别看公式，先看直觉

困惑度最直观的理解是：

模型在每一步预测时，平均相当于在多少个候选词之间摇摆不定。

因此：

- 若模型几乎总能把正确词的概率压得很高，它就不怎么困惑，困惑度就低；
- 若模型总觉得很多词都差不多可能，它就很困惑，困惑度就高。

所以，困惑度不是在量“语法对不对”，而是在量：

模型对真实下一个词到底有多意外。

### 5.3 公式到底在做什么

截图里先写的是平均负对数概率：

$$\frac{1}{n} \sum_{t=1}^n -\log P(x_t \mid x_{t-1}, \dots, x_1).$$

这串式子可以机械地拆成四步。

**第一步：看真实答案的概率。** 在第  $t$  步，模型会对“下一个词是谁”给出很多概率。我们不看它给其他词多少分，只看它给**真实出现的那个词**多少概率。

**第二步：取负对数。** 为什么不直接看概率，而要看  $-\log$ ？因为概率乘起来很快会变得非常小，而对数可以把“连乘”变成“连加”，更容易比较，也更稳定。

而且  $-\log p$  很符合直觉：

- 若  $p$  很大， $-\log p$  很小，说明模型不惊讶；
- 若  $p$  很小， $-\log p$  很大，说明模型很惊讶。

**第三步：对整句取平均。** 一句话里每一步都可以算一个  $-\log p$ 。把它们加起来，再除以长度  $n$ ，就得到“平均每个词让模型惊讶多少”。

**第四步：再取指数。** 最后再做一次指数变换：

$$\text{Perplexity} = \exp\left(\frac{1}{n} \sum_{t=1}^n -\log P(x_t \mid x_{t-1}, \dots, x_1)\right).$$

这一步的作用，是把前面的“平均惊讶程度”重新变回一个更容易解释的尺度，也就是前面说的“平均像是在几个候选词之间犹豫”。

## 5.4 一个真正算得出来的例子

下面只看一句很短的话：

```It is raining outside''.`

假设模型在三个关键位置给出的真实词概率分别是：

- 在看到 It 以后，真实下一个词 is 的概率是 0.8；
- 在看到 It is 以后，真实下一个词 raining 的概率是 0.6；
- 在看到 It is raining 以后，真实下一个词 outside 的概率是 0.5。

那么平均负对数概率就是

$$\frac{-\log 0.8 - \log 0.6 - \log 0.5}{3}.$$

若用自然对数近似，

$$-\log 0.8 \approx 0.223, \quad -\log 0.6 \approx 0.511, \quad -\log 0.5 \approx 0.693.$$

所以平均值约为

$$\frac{0.223 + 0.511 + 0.693}{3} \approx 0.476.$$

再取指数，得到

$$\text{Perplexity} \approx e^{0.476} \approx 1.61.$$

这个 1.61 不该死记成某种神秘常数。它的直觉意思是：

这个模型平均来看，像是在大约 1.6 个有效候选词之间做选择。

这说明它相当确定。

5.5 再看一个差模型

如果另一个模型对同一句话给出的真实词概率只有

$$0.3, \quad 0.2, \quad 0.1,$$

那么对应的 $-\log$ 会大得多，平均值也会更大，最后算出来的困惑度也会明显更高。

这就说明：

- 困惑度低：模型通常把较高概率给到真实词；
- 困惑度高：模型常常对真实词押得不准。

5.6 为什么有人说它像“平均分支数”

这个说法不是严格定义，而是帮助理解的比喻。

如果一个模型在每一步都像是“只有 2 个词我拿不准，但基本就在这两个里选”，那它的困惑度会接近 2。如果它看起来像“总有 50 个词都差不多可能”，那它的困惑度就会大得多。

所以，困惑度可以粗略理解成：

模型平均每一步觉得自己有多少个差不多可能的选择。

5.7 极端情况最能说明问题

- 若模型总把真实词概率给到 1，那么每一步的 $-\log 1 = 0$ ，困惑度就是 1。这表示模型完全不困惑。
- 若模型总把真实词概率给得很小，那么 $-\log p$ 会很大，困惑度也会很大。这表示模型总在猜错边上打转。
- 若模型在一个大小为 V 的词表上总是均匀乱猜，即每个词概率都给 $1/V$ ，那么困惑度就接近 V 。这表示它几乎和“完全没学会”差不多。

5.8 最后一句话记住它

困惑度不是在问：

模型说的话好不好听？

它问的是：

当真实下一个词出现时，模型到底有多惊讶？

模型越不惊讶，困惑度越低；模型越常被真实答案“打脸”，困惑度越高。

6 通过时间反向传播：误差是怎样沿时间倒着传回去的

6.1 为什么普通反向传播在这里不够

在普通前馈网络里，信息只走一遍：

输入 $\rightarrow$ 隐藏层 $\rightarrow$ 输出 $\rightarrow$ 损失。

因此反向传播也只要沿着这条链往回传一次即可。

循环神经网络不一样。它不是一层算完就结束，而是把同一个隐藏状态更新规则反复用在很多个时间步上：

$$h_t = f(x_t, h_{t-1}).$$

这意味着第 t 步的损失，不只依赖当前输入 x_t ，还可能间接依赖更早的 $h_{t-1}, h_{t-2}, \dots$ 。所以，如果最终输出错了，责任不一定只在最后一步，前面很多步都可能有一份。

这正是这张图要解决的问题：

最后时刻的误差，怎样才能追溯回更早的时间步？

6.2 先用一句人话概括 BPTT

所谓“通过时间反向传播”可以先粗暴理解成：

先把 RNN 在时间上展开成一条很长的前馈链，再对这条长链做普通反向传播。

这里“展开”的意思不是改模型，而是为了分析方便，把

$$h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \dots \rightarrow h_T$$

一格一格画开。这样你就能看见：第 T 步的损失，会顺着这条链，一步一步把梯度传回去。

6.3 黑盒视角：BPTT 到底做哪几步

不看细节公式，BPTT 可以分成四步：

1. **前向展开**。依次计算 $h_1, h_2, \dots, h_T$ ，并得到各时刻输出。
2. **计算损失**。比较模型输出和真实标签，得到总损失 L 。
3. **从后往前传误差**。先从最后时刻开始算梯度，再把这份误差通过隐藏状态链条传给更早时刻。
4. **累加共享参数的梯度**。因为每个时间步都用了同一套参数，所以来自各个时刻的梯度都要加起来，最后一起更新参数。

最后这一步很关键。RNN 不是每个时间步有一套新参数，而是同一套参数被重复使用。所以图里那些对 w_h 的求导，会变成“所有时间步贡献的总和”。

6.4 一个三步序列的直观例子

假设我们让模型读一句短话：

```I am not happy''.`

为了简单，只看三个关键时间步：

- 第一步读到 `am`，状态变成  $h_1$ ；
- 第二步读到 `not`，状态变成  $h_2$ ；

- 第三步读到 happy，状态变成  $h_3$ ，并输出情感判断。

如果模型最后把整句话误判成正面，那么我们会得到一个损失  $L$ 。这时反向传播不会只说“最后一步错了”，而会继续追问：

- $h_3$  为什么会导导致错误输出？
- $h_2$  对  $h_3$  有什么影响？
- $h_1$  又怎样通过  $h_2$  间接影响了  $h_3$ ？

所以，梯度流动的直觉图像是：

$$L \longrightarrow h_3 \longrightarrow h_2 \longrightarrow h_1.$$

这就是“通过时间反向传播”的名字来源。它不是只在网络层之间来回传，还要沿着时间轴来回传。

## 6.5 为什么图里会出现递推求导

先说朴素想法。图里真正难的，不是“怎么对一个函数求导”，而是“同一个参数  $w_h$  在很多个时间步都被重复用了，那么最后的损失到底该怎样给它记账”。

为了把问题压缩到最核心，先写一个最简化的 RNN：

$$h_t = f(x_t, h_{t-1}, w_h), \quad o_t = g(h_t, w_o).$$

再把总损失写成

$$L = \frac{1}{T} \sum_{t=1}^T l(y_t, o_t).$$

这里先不要急着看导数，先把这些符号落地：

- $h_t$  是第  $t$  个时间步的隐藏状态，也就是“模型读到当前位置以后，内部记住了什么”；
- $o_t$  是第  $t$  个时间步的输出，可以理解成模型在这一时刻给出的预测分数或预测结果；
- $y_t$  是第  $t$  个时间步的真实标签，也就是正确答案；
- $l(y_t, o_t)$  是单个时间步的损失，衡量“这一时刻的预测  $o_t$  和真实答案  $y_t$  差多远”；
- $T$  是序列长度，也就是这句话一共有多少个时间步；
- $L$  是整条序列的总损失，它把  $T$  个时间步上的损失做了平均。

如果把它翻译回一句人话，就是：

模型每读一步，就更新一次内部状态  $h_t$ ，再给出一个输出  $o_t$ 。这个输出和真实答案  $y_t$  一比较，就得到这一时刻的误差  $l(y_t, o_t)$ 。整句话读完以后，把所有时刻的误差平均起来，就得到总损失  $L$ 。

这里最关键的事实是： $w_h$  不是只在某一个时刻出现一次，而是在每个  $t$  都参与了  $h_t$  的计算。因此， $w_h$  对总损失的影响，天然应该是所有时间步贡献的总和。

不过这还只是直觉，下面给严格推导。

### 第一步：为什么 $\partial L / \partial w_h$ 是时间求和

因为

$$L = \frac{1}{T} \sum_{t=1}^T l(y_t, o_t),$$

所以直接对  $w_h$  求导，就得到

$$\frac{\partial L}{\partial w_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial w_h}.$$

这一步没有任何技巧，只是“求和的导数等于导数的求和”。它说明：

共享参数  $w_h$  的总梯度，是每个时刻那一份梯度加起来。

### 第二步：把单个时刻的梯度再拆开

现在只看求和里的某一项：

$$\frac{\partial l(y_t, o_t)}{\partial w_h}.$$

因为损失先依赖  $o_t$ ，而  $o_t$  又依赖  $h_t$ ，最后  $h_t$  再依赖  $w_h$ ，所以链式法则给出

$$\frac{\partial l(y_t, o_t)}{\partial w_h} = \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial o_t}{\partial h_t} \frac{\partial h_t}{\partial w_h}.$$

前两项通常不太难理解：

- $\partial l / \partial o_t$  说的是“输出错了多少”；
- $\partial o_t / \partial h_t$  说的是“隐藏状态变化一点，会把输出改多少”。

真正麻烦的，就是最后一项

$$\frac{\partial h_t}{\partial w_h},$$

因为  $h_t$  不只是当前一步的函数，它还把以前所有状态都卷进来了。

### 第三步：为什么 $\partial h_t / \partial w_h$ 会满足递推式

从

$$h_t = f(x_t, h_{t-1}, w_h)$$

出发，对  $w_h$  求导。这里要特别注意： $f$  里虽然显式写了  $w_h$ ，但  $h_{t-1}$  本身也隐式依赖  $w_h$ 。因此链式法则会自动分出两部分：

$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}.$$

这就是截图里的核心公式。它的意思非常朴素：



今天的状态  $h_t$  受参数  $w_h$  的影响，一部分来自“今天直接用了  $w_h$ ”，另一部分来自“昨天的状态已经被  $w_h$  影响过，而昨天又影响了今天”。

所以这条递推求导，不是在炫技，而是在认真区分两条责任路径：

- **直接路径：**  $w_h \rightarrow h_t$ ；
- **间接路径：**  $w_h \rightarrow h_{t-1} \rightarrow h_t$ 。

#### 第四步：把递推式展开一次，就看见图里的求和和乘积

为了看清结构，可以记

$$a_t := \frac{\partial h_t}{\partial w_h}, \quad b_t := \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h}, \quad c_t := \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}}.$$

那么上面的递推式就变成

$$a_t = b_t + c_t a_{t-1}.$$

现在继续往前展开一层：

$$a_t = b_t + c_t(b_{t-1} + c_{t-1}a_{t-2}) = b_t + c_t b_{t-1} + c_t c_{t-1} a_{t-2}.$$

再展开一层：

$$a_t = b_t + c_t b_{t-1} + c_t c_{t-1} b_{t-2} + \dots$$

如果继续这样展开到底，就会得到一个显式公式：

$$a_t = \sum_{i=1}^t \left( \prod_{j=i+1}^t c_j \right) b_i + \left( \prod_{j=1}^t c_j \right) a_0.$$

这里约定：当上面的乘积没有任何项时，它的值按惯例记为 1。因此当  $i = t$  时，

$$\prod_{j=t+1}^t c_j = 1,$$

所以求和里的最后一项正好就是  $b_t$ 。

这个公式可以这样读：

- 第  $i$  步的直接贡献是  $b_i$ ；
- 如果这份贡献要从第  $i$  步一路传到第  $t$  步，中间必须依次穿过  $c_{i+1}, c_{i+2}, \dots, c_t$ ，所以前面会乘上一串

$$\prod_{j=i+1}^t c_j;$$

- 若初始状态  $a_0$  也不为零，那么它自己的影响也会沿整条链传到  $t$ ，于是多出最后那一项

$$\left( \prod_{j=1}^t c_j \right) a_0.$$

在很多教学化简里，会把初始状态视为与参数  $w_h$  无关，于是  $a_0 = 0$ 。这时公式进一步简化为

$$a_t = \sum_{i=1}^t \left( \prod_{j=i+1}^t c_j \right) b_i.$$

这时你就能看出图里后面那些“求和里套乘积”的来源了：

- 每个  $b_i$  都代表“第  $i$  步的直接贡献”，所以最后要把很多时间步加起来，这就是**求和**；
- 从更早时刻  $i$  传到当前时刻  $t$ ，中间要一层层经过  $c_{i+1}, c_{i+2}, \dots, c_t$ ，所以会出现一串连乘，这就是**乘积**。

所以，图里复杂的公式并不是凭空冒出来的，而只是下面这句话被彻底写开了：

越早的时间步，想影响今天，就必须穿过越长的隐藏状态链条。

## 6.6 为什么会出现梯度消失和梯度爆炸

现在问题来了。若梯度要从  $T$  一路传回 1，中间就要连续乘很多项，尤其是不断乘

$$\frac{\partial h_t}{\partial h_{t-1}}$$

这样的因子。

如果这句话看起来还是有点跳，我们就把链条缩短，只看三步：

$$h_{t-2} \longrightarrow h_{t-1} \longrightarrow h_t \longrightarrow L.$$

现在问一个非常具体的问题：

最后的损失  $L$  对更早状态  $h_{t-2}$  的梯度，为什么会变成一串乘积？

原因其实只有一个：**链式法则**。

因为  $L$  先依赖  $h_t$ ，而  $h_t$  又依赖  $h_{t-1}$ ， $h_{t-1}$  再依赖  $h_{t-2}$ ，所以

$$\frac{\partial L}{\partial h_{t-2}} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}}.$$

这里每一项都对应链条上的一段：

- $\partial L / \partial h_t$  表示“当前状态  $h_t$  改一点，最终损失  $L$  会改多少”；
- $\partial h_t / \partial h_{t-1}$  表示“前一时刻状态  $h_{t-1}$  改一点，会把当前状态  $h_t$  改多少”；
- $\partial h_{t-1} / \partial h_{t-2}$  表示“再往前一时刻的状态  $h_{t-2}$  改一点，会把  $h_{t-1}$  改多少”。

也就是说，想知道“更早的状态对最终损失有多大影响”，就必须一段一段往后传。每穿过一段，就乘上这一段自己的放大倍数或缩小倍数。

如果再往前多看几步，例如一直追到  $h_1$ ，那么就会得到

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_t} \prod_{k=2}^t \frac{\partial h_k}{\partial h_{k-1}}.$$

这就是前面那句“中间要连续乘很多项”的精确来源。不是因为 RNN 特别神秘，而是因为：

更早的状态想影响最后的损失，必须穿过整条时间链；链式法则要求我们把每一段的影响系数全部乘起来。

**把截图直接翻译成成人话。** 如果只看截图里的那句话，它其实在说一件很具体的事：

当我们想把“最后错了多少”这件事往前追责时，越早的时间步离最终损失越远，因此它的梯度必须一层层穿过后面的隐藏状态。

而每穿过一层，就会乘上这一层自己的局部导数，比如

$$\frac{\partial h_t}{\partial h_{t-1}}.$$

所以：

- 这张图里出现很多  $\partial h_t / \partial h_{t-1}$  这样的项，不是在引入新概念，而是在记录“梯度每往前穿一层，局部会被放大多少或缩小多少”；
- 图里说“中间要连续乘很多项”，意思不是随便乘，而是**沿着时间链，每过一层就按链式法则补乘一次**；
- 因此，时间链越长，这串乘积就越长，梯度就越容易变得极小或极大。

如果把这张图压成一句最短的话，就是：

RNN 里的梯度不是“跳回去”的，而是“沿着时间链一段一段传回去”的；既然是一段一段传，每一段的局部导数就必须全部乘进去。

这时会出现两种极端：

- 若这些因子大多小于 1，连乘很多次后就会越来越小，最后几乎传不到前面去。这叫**梯度消失**。
- 若这些因子大多大于 1，连乘很多次后就会越来越大，最后数值爆掉。这叫**梯度爆炸**。

所以，BPTT 一方面让我们能训练 RNN，另一方面也把 RNN 的核心困难暴露出来了：**时间链太长时，梯度很不稳定。**

## 6.7 为什么实践里常说 truncation 和 detach

既然完整 BPTT 会让链条非常长，实践里常常不会一直回传到序列最开始，而是只回传最近若干步。这叫**截断的 BPTT**。

你可以把它理解成一种工程上的妥协：

我承认更早的历史也有影响，但为了算得动、训得稳，我先只追最近这一段责任。

而 ‘detach’ 的作用，也可以从这个角度理解：它是在某个时间点把梯度链条人为截断，告诉系统“再往前的那部分历史，这次先别继续追责”。

## 6.8 这张图到底在回答什么真实问题

这张图想回答的，不是“BPTT 有几种公式写法”，而是一个非常实际的问题：

如果 RNN 在处理很长的文本时犯了错，我们到底要把这份错误往前追责多远？

想象我们在训练一个字符级语言模型，让它读字符串

the time machine

并且每读到一个字符，就预测下一个字符。

例如，模型刚读完

the time ma

时，应该预测下一个字符是 c。如果它预测错了，那么这个错误理论上可能和很多更早的字符都有关，比如：

- 最近的 a、m 当然有关；
- 更早的 time 也可能有关；
- 再更早的 the 也未必完全无关。

所以训练时真正棘手的问题不是“要不要反向传播”，而是：

这次更新参数时，我是要把责任一路追到最早的字符，还是只追最近几步？

截图上半部分比较的，就是这个问题的三种处理策略。

## 6.9 先把截图里的出场对象翻译成成人话

在这张图里，你可以把整件事想成下面这条流水线：

1. 当前时间步读到一个输入  $x_t$ ；

2. 模型内部保存着一个记忆  $h_{t-1}$ ;
3. 当前输入和旧记忆合在一起，更新成新的记忆  $h_t$ ;
4. 再由  $h_t$  产出当前输出  $o_t$ ;
5. 把  $o_t$  和正确答案  $y_t$  一比较，得到这一时刻的损失;
6. 所有时刻的损失加总或平均后，得到整段序列的总损失  $L$ 。

因此，这里最重要的几个对象分别是：

- $x_t$ ：第  $t$  步读到的输入，比如当前字符或当前词；
- $h_t$ ：第  $t$  步之后模型脑子里记住的内容；
- $o_t$ ：第  $t$  步给出的预测分数；
- $y_t$ ：第  $t$  步的正确答案；
- $L$ ：把整段序列上的错误汇总起来得到的总损失。

现在可以把“反向传播”翻译成一句最简单的话：

当总损失  $L$  很大时，我们要倒着问：到底是哪些更早的状态和输入，把今天的错误一步步传过来的？

## 6.10 最小贯穿例子：为什么会出现“追责多远”的问题

继续用

the time machine

这个例子。

假设模型在读到最后几个字符时，把下一个字符预测错了。那我们更新参数时，有三种态度：

- 第一种态度：既然更早字符也可能有责任，那就全部追到底；
- 第二种态度：只追最近几步，太早的先不管；
- 第三种态度：有时追远一点，有时追近一点，用随机方式折中。

这三种态度，正对应截图里的三种策略。

## 6.11 上半部分：三种策略到底在干什么

### 1. 完整计算。 完整计算的意思是：

这次误差要沿着整条时间链一直往前传，直到序列开头。

如果句子有  $T$  步，那就从最后一步一直追到第 1 步。这样做最彻底，因为任何更早的依赖都不会被漏掉。

但代价也最大：

- 链条很长，计算很慢；
- 梯度要穿过很多步，容易消失或爆炸；
- 训练会变得很不稳定。

### 2. 常规截断。 常规截断的意思是：

我承认更早历史可能有责任，但这次只追最近  $k$  步。

比如  $k = 5$ ，那就只让梯度回传最近 5 步。它牺牲了一部分长期依赖，但换来了更快、更稳的训练。

### 3. 随机截断。 随机截断的意思是：

不要每次都固定追同样长；有时追短一点，有时追长一点，让不同长度的依赖都有机会被看到。

它想解决的是：固定截断太死板，模型总在同样的时间范围里学习；随机截断则试图在效率和长期依赖之间做更柔和的折中。

**4. 图 8.7.1 里的三行到底在画什么。** 那张小图画的不是三种不同的网络，而是三种不同的**梯度回传长度**。图注已经说了，三行从上到下分别是：

- 随机截断；
- 常规截断；
- 完整计算。

因此你可以把它直接读成：

- 上面一行长短不一，表示每次回传的长度是随机的；
- 中间一行一段一段长度差不多，表示每次只回传固定长度；
- 最下面一行是一条完整长链，表示误差一直回传到最早时刻。

这张图最终想表达的核心只有一句：

训练 RNN 时，真正要决定的不是“要不要反向传播”，而是“梯度沿时间轴往前追责多远”。

## 6.12 下半部分：为什么突然写出一组线性公式

当上半部分把“追责多远”这个问题讲清楚以后，下半部分就开始回答另一个问题：

如果我们真的要分析梯度怎样沿时间链传播，应该先把 RNN 写成什么样子，才最容易看清依赖关系？

所以它先写了一个**最简化的线性 RNN**：

$$h_t = W_{hx}x_t + W_{hh}h_{t-1}, \quad o_t = W_{qh}h_t,$$

再写总损失

$$L = \frac{1}{T} \sum_{t=1}^T l(o_t, y_t).$$

先别看推导，先看这三条式子在说什么。这三条式子只是在把“RNN 每一步干什么”写成数学版：

1. 当前输入  $x_t$  提供新信息；
2. 旧状态  $h_{t-1}$  提供历史记忆；
3. 二者合起来得到新状态  $h_t$ ；
4. 新状态  $h_t$  再变成输出  $o_t$ ；
5. 输出  $o_t$  和真实答案  $y_t$  一比，就得到这一时刻的误差；
6. 所有时刻误差平均起来，就是总损失  $L$ 。

这些权重各管什么。

- $W_{hx}$ ：当前输入怎样进入现在；
- $W_{hh}$ ：过去记忆怎样传到现在；
- $W_{qh}$ ：现在的状态怎样变成输出。

这里真正和“时间链”最相关的是  $W_{hh}$ ，因为它控制的是

过去怎样影响现在。

**为什么下半部分要写成线性形式，而不写更复杂的神经网络。** 因为这里的目的是不是让模型最强，而是让**依赖关系最清楚**。

如果一上来就塞进激活函数、门控、复杂结构，读者会先被模型细节淹没，看不清“误差是怎么沿时间链传播的”。所以作者故意选了一个最简单的版本，只为了把骨架露出来。

一个最关键的直觉。 只要把

$$h_t = W_{hx}x_t + W_{hh}h_{t-1}$$

往前代一层，你就会发现：

$$h_t = W_{hx}x_t + W_{hh}W_{hx}x_{t-1} + W_{hh}^2W_{hx}x_{t-2} + \dots$$

这说明什么？

- 越早的输入，想影响今天的状态，就必须穿过更多次  $W_{hh}$ ；
- 反过来，今天的误差想追责到更早的输入，也必须沿着这条时间链一层层穿回去。

所以，下半部分这些公式的最终目的，不是单独介绍一个新模型，而是在为后面的梯度分析铺路：

先把“当前输入怎样和过去记忆合成当前状态”写清楚，后面才能严格分析误差怎样沿时间链倒着传。

### 6.13 再看 8.7.2：这张依赖图到底在算什么

先说真实问题。 上一张图解决的是：

误差要沿时间往前追多远？

而这张 8.7.2 图解决的是另一个更“动手”的问题：

一旦我们决定要做 BPTT，那么参数的梯度到底怎么一项一项算出来？

也就是说，这张图不再讨论“追责多远”，而是在讨论：

既然要追责，那具体从哪儿开始算，哪些量先算，哪些量后算？

先给这一节的路线图。 如果你一看到很多公式就容易迷失，那么最好的读法不是逐项硬看，而是先记住这节只有下面 4 个目标：

1. 先算每个时间步的输出误差  $\partial L / \partial o_t$ ；
2. 再利用输出误差，算最靠近输出层的参数梯度，例如  $\partial L / \partial W_{qh}$ ；
3. 再把责任从输出层传回隐藏状态，得到  $\partial L / \partial h_t$ ；
4. 最后再由隐藏状态的梯度，继续去算更早状态和其余参数的梯度。

所以你可以把这一节理解成一条计算流水线：

先算输出误差 → 再算输出层参数  
 → 再算隐藏状态责任  
 → 最后传给更早时刻和其余参数。

后面的每条公式，都只是这条流水线里的某一个小步骤，而不是毫无目标地突然出现。



**先把图里的出场对象翻译成成人话。** 图里画的是一个只有 3 个时间步的最简 RNN 计算图。你可以从左到右这样读：

- $x_1, x_2, x_3$ ：三个时间步的输入；
- $h_1, h_2, h_3$ ：三个时间步的隐藏状态，也就是模型在每一步记住的内容；
- $o_1, o_2, o_3$ ：三个时间步的输出；
- $y_1, y_2, y_3$ ：三个时间步各自的正确答案；
- $L$ ：把三个时间步的误差汇总起来后的总损失；
- 蓝色方框里的  $W_{hx}, W_{hh}, W_{qh}$ ：三组共享参数。

这里“共享参数”很关键。它的意思不是每个时间步有一套自己的参数，而是：

同一个  $W_{hx}$ 、同一个  $W_{hh}$ 、同一个  $W_{qh}$ ，会在第 1、第 2、第 3 步反复使用。

**最小贯穿例子。** 还是想象字符序列：

$$\mathbf{t} \rightarrow \mathbf{h} \rightarrow \mathbf{e}.$$

在第 1 步，模型读入  $x_1$ ；在第 2 步，读入  $x_2$ ；在第 3 步，读入  $x_3$ 。

每一步都做同样的事情：

$$h_t = W_{hx}x_t + W_{hh}h_{t-1}, \quad o_t = W_{qh}h_t.$$

然后每一步都有一个小误差  $l(o_t, y_t)$ ，最后把三步误差平均起来，得到总损失  $L$ 。

所以，这张图真正展示的是一条完整责任链：

输入  $x_t$  先影响状态  $h_t$ ，状态  $h_t$  再影响输出  $o_t$ ，输出  $o_t$  再影响总损失  $L$ 。

**为什么作者要先画依赖图。** 因为一旦参数共享、时间展开以后，谁依赖谁就不再一眼看清。

例如：

- $W_{qh}$  只直接影响输出层，所以它的梯度相对好算；
- $W_{hh}$  会影响  $h_1, h_2, h_3, \dots$  全部状态，所以它的梯度最麻烦；
- $h_t$  自己也不只是影响当前输出  $o_t$ ，还会继续影响未来状态  $h_{t+1}, h_{t+2}, \dots$ 。

这就是为什么图里要把箭头全画出来：不是为了好看，而是为了让你一眼看到

一个量到底通过哪些路径，最终影响了损失  $L$ 。

**目标 1：先算最容易的一项  $\partial L / \partial o_t$ 。** 每个输出  $o_t$  只直接进入自己的单步损失  $l(o_t, y_t)$ 。所以：

$$\frac{\partial L}{\partial o_t} = \frac{1}{T} \frac{\partial l(o_t, y_t)}{\partial o_t}.$$

这一步很简单，因为  $o_t$  的责任范围很局部：

第  $t$  步的输出错了多少，就看第  $t$  步自己的损失。

**目标 2：再算输出层参数  $\partial L / \partial W_{qh}$ 。** 因为

$$o_t = W_{qh} h_t,$$

所以  $W_{qh}$  对损失的影响，是把每个时间步上的输出误差都收集起来。

直觉上你可以把它理解成：

输出层参数  $W_{qh}$  在每一步都拿当前状态  $h_t$  去生成输出，因此它的总梯度，就是“每一步输出误差”乘上“那一步的状态特征”，再把所有时间步加起来。

所以图里的

$$\frac{\partial L}{\partial W_{qh}}$$

之所以写成一串按时间求和的形式，本质原因只有一个：

同一个  $W_{qh}$  在每个时间步都被重复用了。

**目标 3：把责任从输出层传回隐藏状态，得到  $\partial L / \partial h_t$ 。** 这张图最重要的地方，其实不是  $\partial L / \partial W_{qh}$ ，而是

$$\frac{\partial L}{\partial h_t}.$$

因为隐藏状态  $h_t$  不是只影响一个地方，它至少会影响两件事：

- 当前时刻的输出  $o_t$ ；
- 下一时刻的隐藏状态  $h_{t+1}$ 。

所以，当你问“ $h_t$  对总损失  $L$  的责任有多大”时，答案必须拆成两部分：

- 一部分是它对当前输出  $o_t$  的直接责任；
- 另一部分是它通过未来状态  $h_{t+1}$  间接造成的责任。

这就是图里递推式的真正含义：

$$\frac{\partial L}{\partial h_t} = W_{hh}^\top \frac{\partial L}{\partial h_{t+1}} + W_{qh}^\top \frac{\partial L}{\partial o_t}.$$

你可以把它直接翻译成人话：

第  $t$  步隐藏状态的总责任 = 它对未来造成的责任 + 它对当前输出造成的责任。

**目标 4：有了  $\partial L/\partial h_t$  以后，才能继续往前传。** 因为一旦有了上面这个式子，你就知道该怎样实际往回算：

- 先从最后一个时间步开始；
- 先算最后一步的输出误差；
- 再得到最后一步的隐藏状态梯度；
- 然后用递推式把责任一步步传回更早的  $h_{t-1}, h_{t-2}, \dots$ 。

所以这张图的核心，不是“公式很多”，而是它告诉你一个**计算顺序**：

先算局部容易的梯度，再利用依赖图，把它们沿时间链逐步往前传。

**对比：这张图没有在做什么。** 为了防止误解，再强调两点：

- 这张图不是在介绍一个新网络结构；它只是把已经有的 RNN 画成依赖图。
- 这张图也不是在说“所有梯度都要一次性闭着眼写出来”；它强调的是**按依赖关系分步算**。

**把 8.7.2 压成一句最短的话。**

8.7.2 真正在做的，是把一个时间展开后的 RNN 画成依赖图，然后告诉你：总损失  $L$  对各个参数和隐藏状态的梯度，应该沿哪些路径、按什么顺序一步步算出来。

## 6.14 把整张图压成一句最短的话

这张图从头到尾，其实只在回答一个问题：

RNN 犯错以后，误差要不要追到很早的过去；如果要追，追多远；而为了分析这件事，我们要先把 RNN 写成一个最容易看清时间依赖的形式。

## 6.15 对比：这张图没有在讲什么

为了防止误读，再明确两点：

- 这张图不是在发明三种不同的 RNN 结构；它讲的是**同一个 RNN 在训练时怎样沿时间回传梯度**。
- 下半部分那组线性公式也不是在说“真实 RNN 只能这么简单”；它只是一个方便分析的简化模型。

## 6.16 最后一句话记住 BPTT

如果把这一整节压成一句话，那么 BPTT 就是：

把 RNN 沿时间展开，把最后的误差沿着隐藏状态链条一步步传回更早时刻，并把所有时间步对共享参数的贡献累加起来。

它之所以重要，是因为没有它，RNN 学不会长期依赖；它之所以困难，是因为这条时间链一长，梯度就容易消失或爆炸。

## 7 小结

你可以把这张图压缩成下面两句话。

- 下半部分公式：只是普通前馈网络，做的是“当前输入  $\rightarrow$  中间特征  $\rightarrow$  输出”的静态映射。
- 上半部分公式：才是真正的循环结构，因为它额外保留并更新了一个随时间传播的状态  $h_t$ 。

所以，你没看懂的关键其实不在于矩阵符号本身，而在于要分清两种“hidden”的含义：

- $H$ ：隐藏层特征，是当前输入算出来的中间结果；
- $h_t$ ：隐藏状态，是把过去信息压缩后沿时间传下去的记忆。

一旦这一区分清楚，整张图就清楚了：下半部分是在回顾普通前馈网络，上半部分是在说明为什么序列任务需要比前馈网络多出一个“可递推的记忆状态”。